

Microstructure ML: Predicting Short-Horizon Price Direction from Limit Order Book and Trade Data

Charles Heese

DS 4400: Machine Learning and Data Mining I
Spring 2026

Abstract

This project studies short-horizon direction prediction using high-frequency market microstructure data from the Optiver Realized Volatility Prediction dataset. The public data set is converted into a binary classification problem by constructing custom leakage-safe labels based on whether the future average midprice is above or below the current average midprice. From the first 300 seconds of each time window, 59 were engineered from both the limit order book and trade stream, including spread, depth, imbalance, weighted average price, midprice statistics, trade-price summaries, and early-versus-late shift features.

To keep evaluation leakage-safe, the data was split by `time_id` using a `GroupShuffleSplit`, computed the move-filtering threshold on the training set only, and then apply the same threshold to both train and test. Five models were compared under a shared final horizon setting: a dummy majority baseline, logistic regression, random forest, histogram gradient boosting, and an MLP classifier.

The shared horizon is selected by logistic regression during a horizon search and corresponds to `future_start = 300` and `future_width = 10`. The best final model is the MLP classifier, which achieves 0.8117 accuracy, 0.8116 F1, 0.8958 ROC AUC, and 0.8937 average precision on the held-out test set. Across experiments, shorter prediction horizons are consistently better than longer ones, and both trade-based and order-book-based features contribute meaningful signal. Held-out permutation importance shows that recent trade prices and short-term midprice movement features are the strongest predictors while stock identity contributes very little. Overall the project shows that a carefully engineered, leakage-safe feature pipeline can predict very short-horizon price direction significantly better than a naïve baseline.

1 Problem Description and Motivation

1.1 Task Definition

The machine learning problem in this project is binary classification. For each `stock_id` and `time_id` window, the goal is to predict whether the average midprice moves up or down relative to the current average midprice. The current price is summarized over the anchor interval from 240 to 300 seconds, and the future price is summarized over a later interval defined by `future_start` and `future_width`. After computing the relative move, the final label is 1 if the move is positive and 0 otherwise.

This framing turns the original Optiver dataset into a directional prediction task. Instead of using the competition’s realized volatility target, the project constructs its own classification labels directly from the order book data. That makes the problem more aligned with short-horizon trading and execution decisions, where the main question is often not how volatile the next interval will be, but whether the next move is more likely to be upward or downward.

1.2 Why This Problem Matters

Short-horizon price direction is important because even small predictive edges can be useful in electronic trading environments. In practice, market makers and execution algorithms constantly make decisions about whether to quote more aggressively, whether to submit passive or active order, and how to reduce adverse selection. If a model can identify a small but reliable directional tendency over the next few seconds, that information can potentially improve execution quality and inventory management.

This problem is also interesting from a machine learning perspective because the underlying signal is weak, noisy, and highly local in time. The data is high-dimensional, event driven, and structured by both book state and trade activity. That makes it a good test case for feature engineering leakage-safe evaluation, model comparison, and interpretability analysis.

1.3 Related Work

DeepLOB is one of the most relevant prior papers because it uses deep neural networks on limit order book data to predict short-term price movement. Its main takeaway is that local order book structure contains predictive information, especially when modeled with architectures that capture spatial and temporal patterns. That paper motivates treating order book state as a predictive object rather than using only simple summary statistics.

Sirigano and Cont emphasize that high-frequency formation exhibits recurring local patterns across many assets. That idea is relevant here because the project pools information across 112 stocks and asks whether general short-horizon predictive structure exists in engineered microstructure features. Their work supports the idea that common market microstructure behavior can be learned statistically.

2 Dataset

2.1 Dataset Source and Raw Structure

The project uses the public Optiver Realized Volatility Prediction dataset from Kaggle. The raw data is split by `stock_id` and contains two main sources. The first is the order book data, which includes bid and ask prices and sizes at the top of two levels of the book for many seconds within each `time_id` window. The second is trade data, which includes executed trade price, trade size, `order_count`, and `seconds_in_bucket`.

Each `stock/time_id` pair can be treated as a short market episode. The raw book and trade records occur at the second level within each episode, so the first step of the project was to aggregate them into a single feature row per `stock_id` and `time_id`. Before filtering, the full engineered table contains 428,835 windows across 112 unique stocks.

The final table has 65 columns total, of which 59 are modeling features after excluding targets and label-construction columns. Of those 59 features, 58 are numeric and 1 is categorical, namely `stock_id`.

2.2 Label Construction

Labels are built directly from the order book data using the midprice, defined as the average of `bid_price1` and `ask_price1`. For each `time_id`, the current midprice the mean over the anchor interval [240, 300). The future midprice is the mean midprice over a later interval [`future_start`, `future_start` + `future_width`). The relative move is then computed as: $\text{relative move} = (\text{future}$

midprice – current midprice) / current midprice.

A raw target of 1 is assigned if the relative move is positive, and 0 otherwise. However, the final modeling task removes the smallest absolute moves because those examples are the noisiest and least financially meaningful. To do this without leakage, the threshold is computed from the training set only as the 33rd percentile of the absolute relative move. That threshold is then applied to both train and test, and the final binary target is created after filtering.

This label design is one of the key original components of the project because it repurposes the volatility dataset into a short-horizon directional classification problem while preserving a clean separation between observation data and future target information.

2.3 Dataset Statistics

The final modeling pipeline produces 229,856 filtered training examples and 57,342 filtered test examples. The filtered class balance is close to even, with a positive rate of 50.75 percent in training and 50.08 percent in testing. This is useful because it means the strong results are not coming from trivial class imbalance.

Feature coverage is also clean. The final pipeline contains 36 book-derived features, 22 trade-derived features, and 1 categorical stock identifier. There are 8 shift features that compare early and late portions of the observation window. After filling in trade-related missing values and using preprocessing pipelines, there are effectively no missing values remaining in the modeling table.

Table 1: Dataset statistics for the final modeling pipeline.

Statistic	Value
Full unfiltered windows	428,835
Filtered train windows	229,856
Filtered test windows	57,342
Total columns before dropping labels	65
Model features used	59
Numeric features	58
Categorical features	1
Unique stocks	112
Train positive rate	50.75%
Test positive rate	50.08%
Features with missing values	0
Overall feature missing percent	0.00%
Maximum feature missing percent	0.00%
Book-derived features	36
Trade-derived features	22
Shift features	8
Highly correlated numeric pairs ($ r \geq 0.95$)	51

2.4 Feature Definition and Feature Groups

The feature set is designed to capture three main things: liquidity state, recent price behavior, and recent trading activity. Book features include spread, relative spread, depth, imbalance, weighted average price, midprice statistics, and early/late summaries. Trade features include volume, trade count, order count, trade-price summaries, dollar volume, and early/late shift measures.

Stock identity is included as a categorical variable so the models can learn stock-specific patterns if those are useful.

One important insight from the final tables is that the feature set is not dominated by a single source. Later analysis shows that both book and trade features matter. At the same time, stock_id has almost no importance in the held-out permutation analysis, which suggests that the predictive signal is mostly coming from local microstructure conditional rather than memorizing specific stocks.

Table 2: Feature group counts in the final modeling table.

Feature group	Shift feature?	Count
book	No	31
book	Yes	5
stock_id	No	1
trade	No	19
trade	Yes	3

Table 3: Representative engineered features used in the final pipeline.

Feature / Group	Description
spread1_mean	Mean top-of-book bid-ask spread over the observation window.
rel_spread1_mean	Mean relative spread, computed as spread divided by the mid-price.
imbalance1_mean	Mean level-1 order book imbalance, measuring buy-side versus sell-side depth.
depth1_mean	Mean total level-1 depth, computed from bid size 1 plus ask size 1.
wap1_mean	Mean weighted average price at level 1 using bid and ask sizes as weights.
obs_mid_return	Midprice return over the observation window from the first to last observed midprice.
mid_shift	Difference between late-window and early-window mean mid-price.
last_trade_price	Last executed trade price observed before prediction begins.
trade_count	Number of trades in the observation window.
volume_shift	Change in traded volume between the late and early subwindows.
stock_id	Categorical stock identifier encoded with one-hot encoding.

2.5 Exploratory Data Analysis

The EDA suggests that the filtered classification problem is reasonably balanced. The label balance figure shows that the up and down classes are very close in frequency after removing the smallest absolute price moves. That means metrics like accuracy and F1 are meaningful and not being driven by a heavily skewed target.

The spread histogram shows a highly right-skewed distribution. Most windows have relatively small average spreads, but there is a tail of wider-spread windows. This fits the idea that liquidity conditions vary substantially across market states and may help explain why some windows are harder to predict than others.

The order book imbalance distributions for the two classes overlap heavily and are both centered near zero. That means that the imbalance alone is not enough to separate upward and downward future moves. However the slight shape differences and the fact that imbalance-related variables still appear in the engineered features suggest that it can still be useful when combined with other signals.

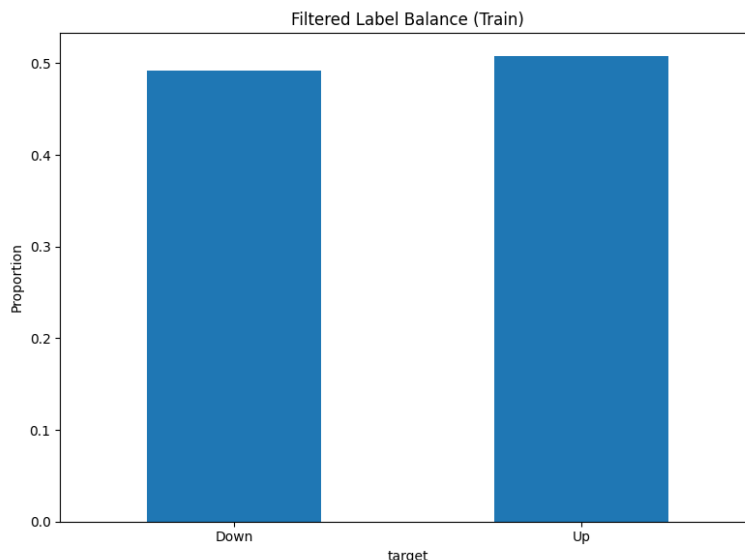


Figure 1: Filtered training-set label balance after removing the smallest absolute price moves.

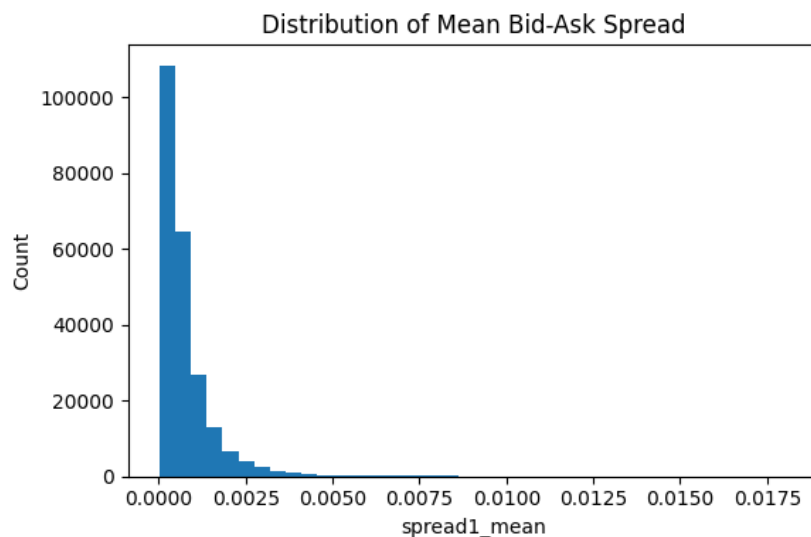


Figure 2: Distribution of the mean bid-ask spread.

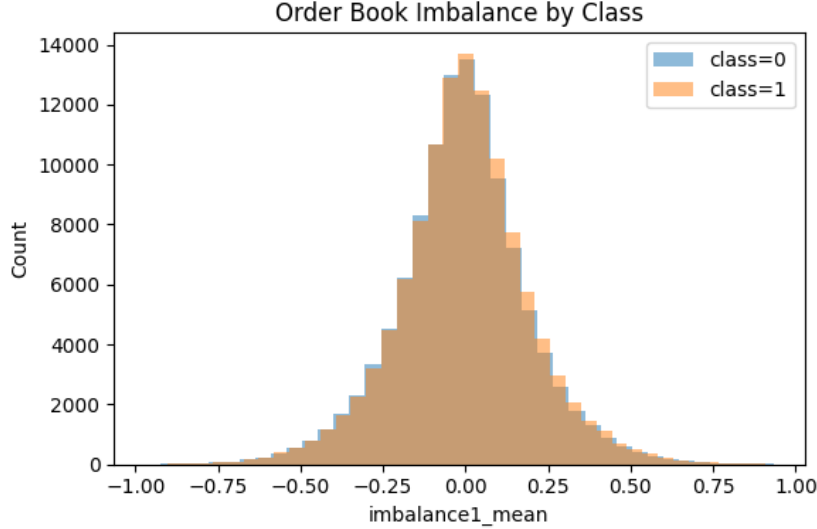


Figure 3: Order book imbalance distribution by class.

3 Approach and Methodology

3.1 Leakage-Safe Experimental Pipeline

The overall pipeline is designed to prevent leakage at every stage. First, features are computed only from the observation portion of the window, which uses `seconds.in_bucket` less than 300. Second, labels are built from future price information that occurs later in the same `time_id` window. Third, the data is split using `GroupShuffleSplit` with `time_id` as the grouping variable, so the same `time_id` split is not across train and test.

Using `time_id` as the grouping variable is very important because the same `time_id` exists across many stocks and reflects common market time conditions. If some stocks from the same `time_id` were in train and others were in test, the model could partially benefit from shared market-state information. Grouped splitting avoids that problem.

After the grouped split, the project computes the move threshold from the training set only and uses that same threshold to filter both train and test. This is another import leakage-control step. If the threshold were chosen on the full dataset, then test information would influence label construction. By keeping threshold selection inside training data only, the final evaluation remains cleaner.

3.2 Feature Engineering

3.2.1 Book Features

The book feature block begins with direct measures of liquidity and book shape. These include `spread1`, `spread2`, relative spread, depth at the first two levels, level-1 imbalance, and weighted average price. Midprice is also computed from the top-of-book bid and ask prices and is used to derive several additional statistics.

The code then aggregates these variables over the observation window to produce summary statistics such as means, standard deviations, minima, maxima, last values, and counts of updates. In addition to global summaries, the observation window is split into an early half and a late half,

and separate means are computed for several book variables. Shift features are then created by subtracting the early summaries from the late summaries.

the early-versus-late structure is meant to capture short-term changes in the state of the book rather than only its average level. For example, if `mid_shift` and `spread1_shift` are intended to indicate whether the market state is becoming more directional or more stressed even before the prediction interval begins.

3.2.2 Trade Features

The trade feature block summarizes executed market activity rather than posted liquidity. The pipeline computes total volume, mean and standard deviation of the trade size, trade count, total order count, mean and standard deviation of the trade price, dollar volume, last trade price, and the number of distinct trade seconds.

Just like with the book data, the trade data is split into early and late sub windows. This produces early and late summaries for volume, trade count, mean trade size, and dollar volume. The pipeline then creates several shift features such as `volume_shift`, `trade_count_shift`, and `mean_trade_size_shift`.

These trade features matter a lot in the final results. In the permutation-importance tables, `last_trade_price` and `mean_trade_price` are among the most important predictors for multiple models.

3.2.3 Data Cleaning and Missing Values

Trade-based features can be missing for windows with no trades. Instead of leaving these as missing values, the code explicitly fills the trade aggregate columns with zeroes after merging them into the main feature table. This is sensible because a window with no trades is in and of itself informative and should not necessarily be discarded.

After feature construction, numeric columns are checked for infinite values and those are converted to missing values before model preprocessing. Inside the pipelines, numeric features are median-imputed, and the categorical `stock_id` is imputed with the most frequent value before one-hot encoding. The final summary tables show that the effective feature matrix has no remaining missing values.

3.3 Preprocessing

Different preprocessing pipelines are used depending on the model class. Logistic regression and the MLP use median imputation followed by `StandardScaler` on the numeric variables, since both models are sensitive to feature scale. `Stock_id` is one-hot encoded.

For random forest, numeric features are median-imputed but not scaled, since tree-based models do not require standardized inputs. `Stock_id` is still one-hot encoded. Histogram gradient boosting uses a similar structure, but the categorical one-hot block is made dense because that estimator expects dense inputs.

This preprocessing design lets each model see the same engineered feature set while keeping the input format appropriate for the algorithms.

3.4 Model Definitions

The baseline model is a dummy majority classifier that always predicts the most frequent class. This provides a lower bound and confirms that the task is nontrivial.

The first real baseline is logistic regression with `class_weight` set to `balanced` and `max_iter` set

to 3000. Logistic regression is useful here because it is fast, strong on tabular data, and easy to interpret.

The rest of these were all tuned to their best settings as described below. The random forest model uses 400 trees, `max_depth = 14`, `min_sample_leaf = 15`, `balanced_subsample` class weights, and `random_state = 42`. This model is meant to test whether nonlinear tree interactions can improve over the linear baseline.

The histogram gradient boosting model uses `learning_rate = 0.05`, `max_depth = 6`, `max_iter = 250`, and `min_samples_leaf = 50`. This is a stronger nonlinear boosted-tree baseline that can often perform well on structured tabular data.

The neural network model is an `MLPClassifier` with hidden layers of size (64, 32), `alpha = 1e-4`, `batch_size = 1024`, `early_stopping = True`, `max_iter = 50`, `n_iter_no_change = 5`, and a random state 42. This gives the project a neural-network while still staying practical for the dataset size.

3.5 Evaluation Metrics

The report compares models using accuracy, precision, recall, F1, balanced accuracy, specificity, ROC AUC, average precision, and log loss. Using several metrics is important because no single metric tells the whole story.

Accuracy and F1 summarize threshold classification quality. Precision and recall help to show the tradeoff between false positives and false negatives. Balanced accuracy and specificity are useful even though the filtered classes are nearly balanced, because they make the false-positive side easier to inspect. ROC AUC and average precision evaluate ranking quality rather than only hard predictions. Log loss provides another view by penalizing poorly calibrated confidence scores. Together, these metrics give a complete picture of performance, generalization, and model behavior.

4 Results

4.1 Horizon Selection

Before comparing all models, the project runs a horizon search using logistic regression as the selection model. The grid includes `future_start` values of 300, 330, 360, 420, 480, and 540 as well as `future_width` values of 10, 15, 20, 25, and 30. Logistic regression is used for the search so that the shared final setting is chosen in a simple and stable way before evaluating the full model set.

The best horizon found in this search is `future_start = 300` and `future_width = 10`. Under that setting, logistic regression achieves 0.8070 accuracy, 0.8051 F1, 0.8710 ROC AUC, and 0.8553 average precision on the test set.

The heatmap shows a very clear trend: performance steadily degrades as either `future_start` or `future_width` increases. This suggests that the predictive information in the engineered features is strongest for the shortest horizons and fades as the prediction target moves farther into the future or becomes more averaged across time.

Table 4: Chosen shared horizon setting from the logistic-regression horizon search.

Model	Split	future_start	future_width	n_{train}	n_{test}	Features	Accuracy	F1	ROC AUC	Avg. Precision
logistic_regression	test	300	10	224,756	56,116	59	0.8070	0.8051	0.8710	0.8553

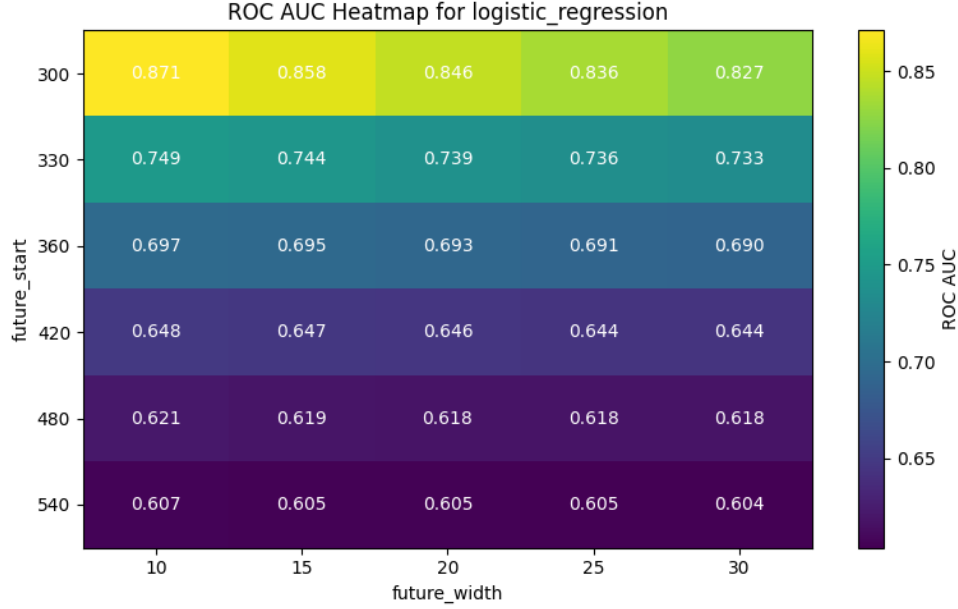


Figure 4: ROC AUC across candidate future_start and future_width settings for logistic regression.

4.2 Final Model Comparison

After fixing the shared horizon at 300/10, all the final models are trained and evaluated under that same setting. The best overall model is the MLP classifier. It achieves 0.8117 accuracy, 0.8033 precision, 0.8200 recall, 0.8116 F1, 0.8118 balanced accuracy, 0.8958 ROC AUC, 0.8937 average precision, and 0.4068 log loss.

Histogram gradient boosting is the second-best overall model. Its results are very close to the MLP, with slightly lower ROC AUC and F1 but slightly higher specificity. Logistic regression remains a strong baseline and is only slightly behind the top two models in accuracy and F1, although its ranking metrics are noticeably lower than those of the MLP and histogram gradient boosting.

Random forest performs substantially better than the dummy baseline but is clearly weaker than the top three models. The dummy majority classifier confirms that the task is meaningful: its accuracy is only 0.4946 and its ROC AUC is exactly 0.5000, which is chance-level ranking performance.

One useful final takeaway is that the nonlinear models do improve over logistic regression, but the gains are model rather than dramatic. That means a large part of the predictive signal is already captured by the engineered features themselves.

Table 5: Final test-set comparison of all models under the shared horizon setting ($future_start = 300$, $future_width = 10$).

Model	Accuracy	Precision	Recall	F1	Balanced Accuracy	Specificity	ROC AUC	Average Precision	Log Loss
mlp_classifier	0.8117	0.8033	0.8200	0.8116	0.8118	0.8036	0.8958	0.8937	0.4068
hist_gradient_boosting	0.8070	0.8046	0.8053	0.8050	0.8070	0.8087	0.8905	0.8874	0.4206
logistic_regression	0.8070	0.8042	0.8060	0.8051	0.8070	0.8079	0.8710	0.8553	0.4780
random_forest	0.7469	0.7434	0.7456	0.7445	0.7469	0.7481	0.8397	0.8390	0.5509
dummy_majority	0.4946	0.4946	1.0000	0.6618	0.5000	0.0000	0.5000	0.4946	6.9826

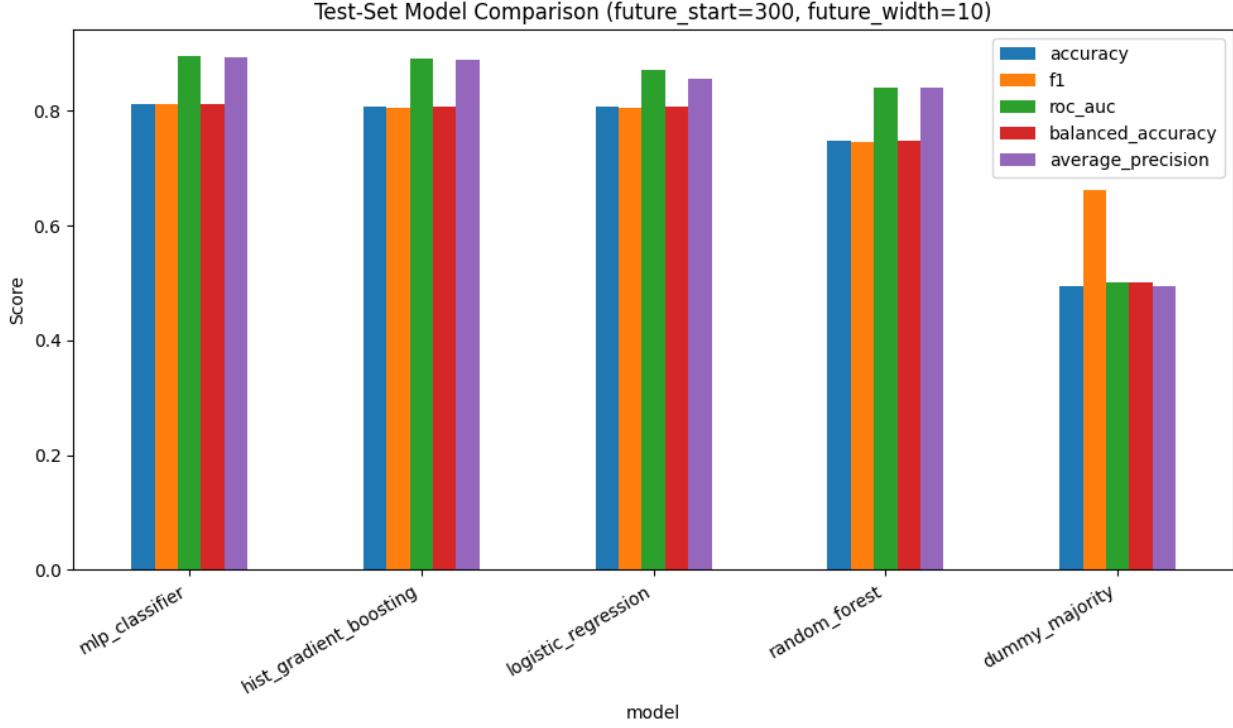


Figure 5: Test-set metric comparison across the final models.

4.3 Error Analysis

The confusion matrices for the top three models show that the MLP makes the most positive predictions and thus catches the truest upwards moves. It records 22,758 true positive and only 4,996 false negatives, which is the strongest recall performance among the top models. The tradeoff is that it also produces 5,571 false positives, which is slightly more than the other two models.

Histogram gradient boosting is more conservative. It has the fewest false positives at 5,427 and highest specificity at 0.8087, but that comes with more false negatives, specifically 5,403. Logistic regression sits very close to histogram gradient boosting and behaves like a middle ground between the boosted tree and the MLP.

This indicates that the best model depends a little on what kind of mistake is considered more costly. If missing upward moves matters more, the MLP is preferable. If reducing false alarms matters slightly more, histogram gradient boosting looks attractive. Since the MLP is also the best on most overall metrics, it is the most natural final choice.

Table 6: Test-set confusion matrix counts for the top three models.

Model	TN	FP	FN	TP	Specificity	Recall	Precision
mlp_classifier	22,791	5,571	4,996	22,758	0.8036	0.8200	0.8033
hist_gradient_boosting	22,935	5,427	5,403	22,351	0.8087	0.8053	0.8046
logistic_regression	22,914	5,448	5,384	22,370	0.8079	0.8060	0.8042

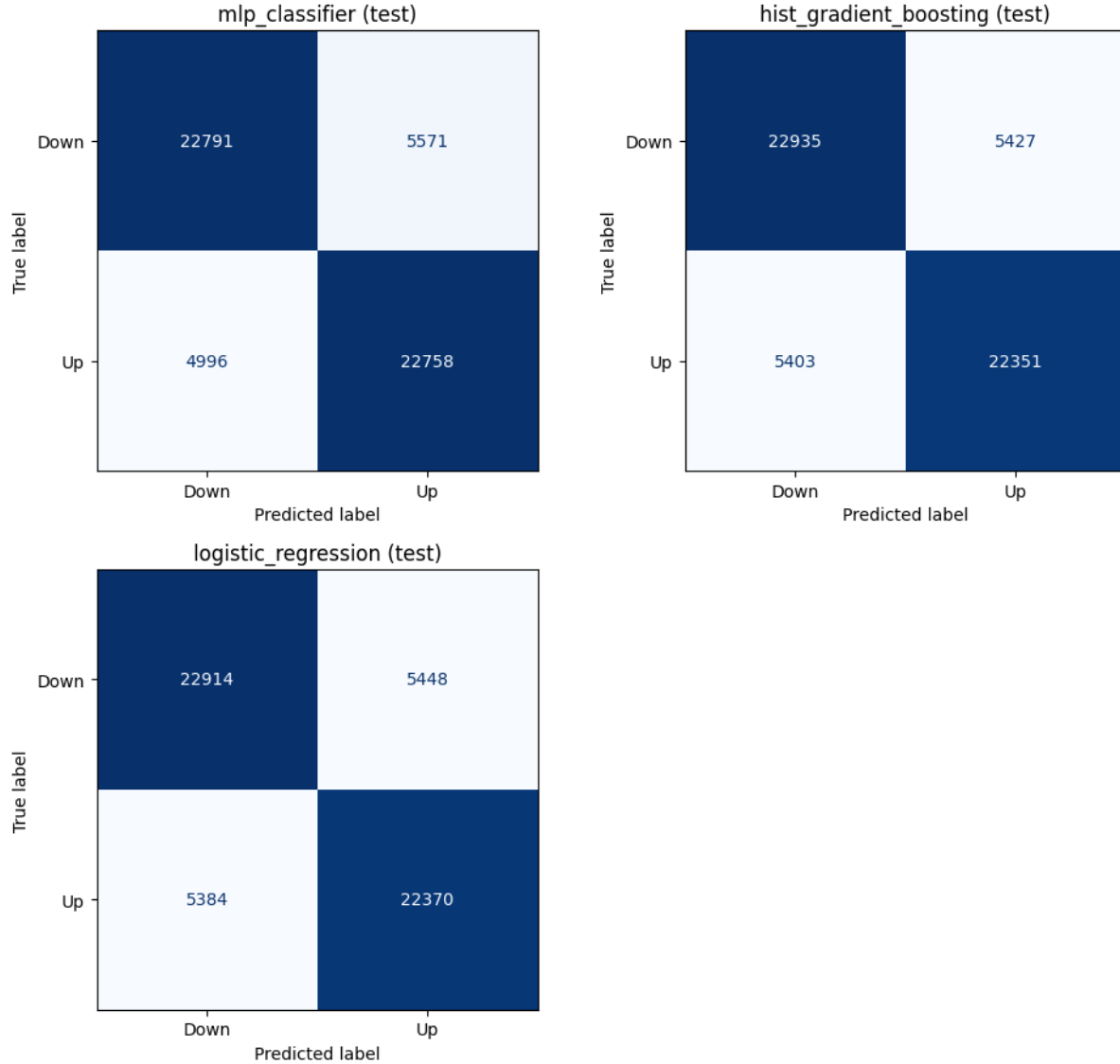


Figure 6: Test-set confusion matrices for the top three final models.

4.4 Generalization and Train-Test Gap

The train-test gap table shows that logistic regression generalizes the most cleanly. Its accuracy and ROC AUC gaps are essentially zero, and its F1 gap is very small. This suggests that the linear model is quite good at creating a robust baseline signal without much overfitting.

The MLP and histogram gradient boosting also generalize fairly well. Their train-test gaps are small across accuracy, F1, ROC AUC, and average precision, which suggests that their performance gains are real rather than purely due to overfitting. Between those two, histogram gradient boosting has slightly smaller gaps, while the MLP still has the better absolute test performance.

Random forest shows the largest generalization gap by far. Its ROC AUC is about 0.059 and its F1 gap is about 0.052, which strongly suggests some overfitting. This is consistent with the final comparison table, where random forest performs much worse on the test set than its training performance would suggest.

Table 7: Train-test metric gaps by model (train minus test). Smaller values indicate better generalization.

Model	Gap Accuracy	Gap F1	Gap ROC AUC	Gap Balanced Accuracy	Gap Avg. Precision
mlp_classifier	0.0074	0.0102	0.0083	0.0072	0.0119
hist_gradient_boosting	0.0058	0.0094	0.0057	0.0058	0.0090
logistic_regression	-0.0007	0.0027	-0.0008	-0.0007	0.0034
random_forest	0.0476	0.0518	0.0589	0.0476	0.0653
dummy_majority	0.0088	0.0079	0.0000	0.0000	0.0088

5 Interpretability and Novel Contributions

5.1 Interpretability

Held-out permutation importance gives a consistent picture across models. For the MLP, the most important feature is `last_trade_price`, followed by `mid_shift`, `obs_mid_return`, `mid_late_mean`, and `mean_trade_price`. This indicates that the strongest signal comes from very recent trade prices and short-term price movement within the observation window, rather than from static liquidity variables alone.

The group-level permutation summary shows that both book and trade features are important for all non-dummy models. For the MLP, the total book-group importance is 0.7640 and the total trade-group importance is 0.5655, while `stock_id` contributes only 0.0045. Similar patterns appear for logistic regression and histogram gradient boosting. This implies that the models are mostly learning from market-state variables rather than memorizing stock identity.

The logistic regression interpretability tells a similar story. Its top features also include `last_trade_price`, `mean_trade_price`, `mid_shift`, `obs_mid_return`, and `mid_late_mean`. That agreement between a linear model and the nonlinear models makes the interpretation more convincing.

Table 8: Held-out permutation importance summed by feature group.

Model	Feature Group	Importance Mean
hist_gradient_boosting	book	0.4525
hist_gradient_boosting	trade	0.3124
hist_gradient_boosting	stock_id	0.0001
logistic_regression	book	0.6315
logistic_regression	trade	0.5421
logistic_regression	stock_id	0.0003
mlp_classifier	book	0.7640
mlp_classifier	trade	0.5655
mlp_classifier	stock_id	0.0045
random_forest	book	0.3419
random_forest	trade	0.2189
random_forest	stock_id	-0.0003

Table 9: Top held-out permutation-importance features for the best final model (MLP).

Feature	Group	Shift Feature?	Importance Mean	Importance Std.
last_trade_price	trade	No	0.3144	0.0019
mid_shift	book	Yes	0.1526	0.0016
obs_mid_return	book	No	0.1469	0.0023
mid_late_mean	book	No	0.1291	0.0019
mean_trade_price	trade	No	0.0899	0.0028
wap1_mean	book	No	0.0666	0.0014
trade_count_late	trade	No	0.0435	0.0024
mid_mean	book	No	0.0355	0.0018
n_trade_seconds	trade	No	0.0297	0.0014
mid_early_mean	book	No	0.0297	0.0008

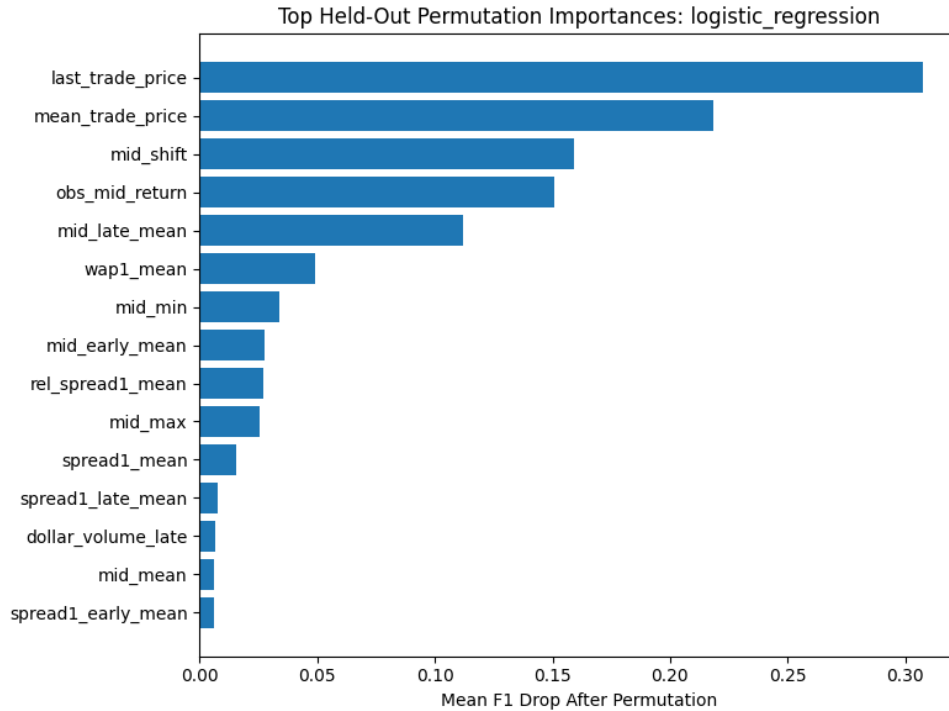


Figure 7: Top held-out permutation importances for logistic regression.

5.2 Novel Contributions

One original contribution of the project is the label design itself. The Optiver dataset was originally framed as a volatility regression problem, but this project converts it into a short-horizon directional classification problem using custom anchor and future windows. That reframing is central to the whole project and is not just copied from the competition setup.

A second contribution is the leakage-safe move-filtering procedure. Instead of using all price moves, the project removes the smallest one-third of absolute relative moves using a threshold computed on the training set only. This creates a cleaner binary prediction target while still respecting proper evaluation boundaries.

A third contribution is the feature design. The project does not just compute static book and trade summaries; it also adds early-versus-late sub window features and shift variables that try to capture short-term acceleration or changes in market state before prediction begins.

Finally, the project goes beyond raw model comparison by adding horizon search, ablation testing, and held-out permutation importance. The ablation study is especially useful because it shows that combining book and trade features is clearly better than using either alone, while the stock’s identity adds almost nothing and shift features contribute little to the logistic regression baseline.

5.3 Ablation Results

The ablation study provides a more detailed view of what matters in the feature set. The full logistic regression model achieves 0.8710 ROC AUC. Removing the stock_id feature slightly improves performance rather than hurting it, which suggests that identity-specific information is not important once the local microstructure features are present.

Removing the shift features has almost no effect on logistic regression. The test ROC AUC changes by essentially nothing. That means the specific early-versus-late shift variables do not add much extra linear signal beyond the rest of the engineered feature set, at least for this model.

In contrast, restricting the model to book-only features drops ROC AUC, and restricting it to trade-only features drops it further. This is a strong result because it shows that both feature sources matter and the best performance comes from combining them.

Table 10: Ablation study for logistic regression under the shared horizon setting.

Ablation	Features	Test Accuracy	Test F1	Test ROC AUC	Test Balanced Accuracy	Test Avg. Precision	ROC AUC Drop vs. Full
full	59	0.8070	0.8051	0.8710	0.8070	0.8553	0.0000
book_only	37	0.7461	0.7444	0.8112	0.7461	0.7914	0.0598
trade_only	23	0.7183	0.7151	0.7753	0.7183	0.7498	0.0957
without_shift_features	51	0.8070	0.8051	0.8710	0.8070	0.8553	0.0000
without_stock_id	58	0.8073	0.8054	0.8712	0.8073	0.8555	-0.0002

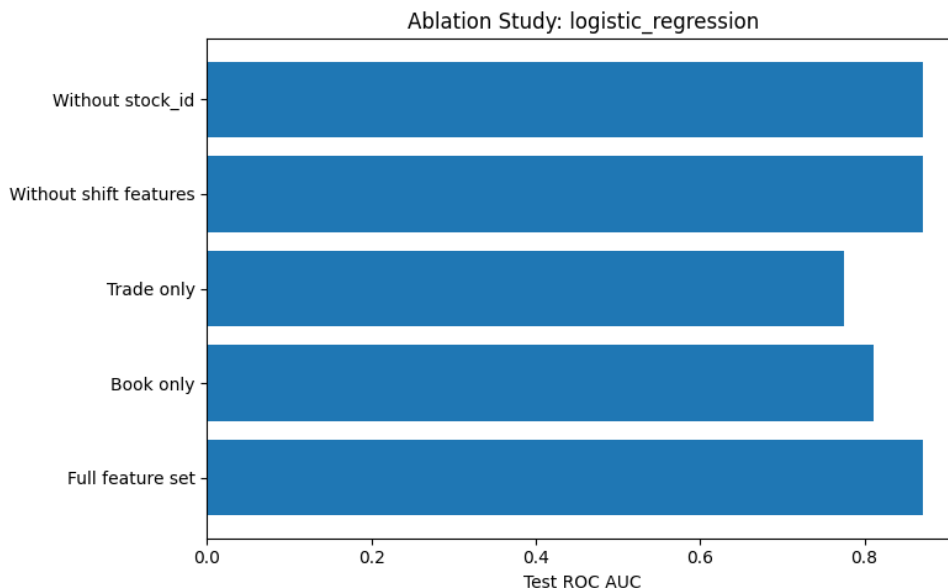


Figure 8: ROC AUC results for the logistic-regression ablation study.

6 Limitations and Future Work

6.1 Limitations

One limitation is that the evaluation is still based on a single grouped train-test split rather than repeated cross-validation over time blocks. This is acceptable as a proof of concept, but more extensive validation would make the conclusions stronger.

A second limitation is that the project focuses on predictive metrics and does not model transaction costs, slippage, queue positions, or market impact. In practice, a model that predicts very small, short-horizon moves well might still not be tradeable after costs.

Another limitation is that the neural network remains a relatively simple tabular MLP rather than a sequence model designed specifically for order book structure. Since papers like DeepLOB use richer temporal architectures, there is room to explore whether sequence-aware models could capture additional signals that the current feature-based pipeline misses.

6.2 Future Work

Future work could include time-based cross-validation, probability calibration, explicit cost-aware evaluation, more systematic hyperparameter tuning, and sequence models that operate directly on the raw book and trade updates instead of only aggregated features.

7 Team Member Contribution

This is an individual project.

Charles Heese completed the full pipeline, including dataset setup, feature engineering for book and trade data, custom label construction, leakage-safe train-test splitting, move filtering, model training, model evaluation, horizon search, ablation study, permutation-importance analysis, figure generation, and report writing.

8 Conclusion

This project shows that a leakage-safe, feature-engineered pipeline can predict very short-horizon price direction from public limit order book and trade data substantially better than chance and substantially better than a dummy baseline. The best final model is the MLP classifier, which achieves 0.8117 accuracy, 0.8116 F1, and 0.8958 ROC AUC on the held-out test set.

The results also tell a consistent story. Shorter horizons are easier to predict than longer ones. Recent trade prices and recent midprice movement are the strongest signals. Both book-derived and trade-derived features are useful, and combining them clearly outperforms using either source alone. Stock identity contributes almost nothing once the microstructure features are present.

Overall, the project succeeds in building a strong and interpretable baseline for short-horizon directional prediction. It also sets up several clear next steps for future work, especially around more advanced neural architectures and more realistic trading-oriented evaluation.

9 AI Disclosure

AI was used in a limited way for code debugging assistance during this project. All major project decisions, including problem formulation, feature engineering, model development, experimentation, interpretation of results, and written analysis, were completed by me.

References

- [1] Optiver Realized Volatility Prediction, Kaggle competition dataset.
- [2] Zhang, Zohren, and Roberts. *DeepLOB: Deep Convolutional Neural Networks for Limit Order Books*.
- [3] Sirignano and Cont. *Universal Features of Price Formation in Financial Markets*.
- [4] Cont, Kukanov, and Stoikov. *The Price Impact of Order Book Events*.